

Subject Name: OBJECT ORIENTED PROGRAMMING DESIGN

Subject Code: CS T33

2 MARKS AND 11 MARKS FOR ALL UNITS

UNIT I

Introduction to Object-Oriented Programming: Evolution of programming methodologies – Disadvantages of conventional programming – programming paradigms – key concepts of object – oriented programming – advantages of OOP – usage of OOP.

Input and output in C++ : Limitations of C – Introduction to C++ – Structure of the C++ program – stream classes – formatted and unformatted data – unformatted console I/O operations – Bit fields, Manipulators – Manipulators with multiple parameter

Control structures: Decision making statements – jump statement – switch case statement – looping statements.

Classes and objects: Defining member functions – rules of inline functions – data hiding or encapsulation – classes – objects and memory – static object – array of objects – objects as function arguments, friend functions, member functions and non-member functions – overloading member functions.

Functions in C++ : Passing arguments – LValues and RValues – return by reference – default arguments – inline functions – function overloading.

PART A – 2 MARK QUESTIONS

1. What are the main characteristics of OOP language?(May 2007)

The important characteristics of object oriented programming are as follows:

- OOP pays more importance to data than to function.
- Programs are divided into classes and their member functions.
- New data items and functions can be comfortably added whenever essential.
- Data is private and prevented from accessing external functions
- Objects can communicate with each other through functions.

2. What are the basic concepts of OOPS?

The basic concepts of OOPS are,

- i. Objects.
- ii. Classes.
- iii. Data abstraction and Encapsulation.
- iv. Inheritance.
- v. Polymorphism.
- vi. Dynamic binding.
- vii. Message passing.

3. Define object.

An object is basic run-time entity in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program has to handle. Each object has the data and code to manipulate the data and these objects interact with each other.

4. What is a class?

A class is a collection of objects of similar type. Once a class has been defined, we can create any number of objects belonging to the class. Class is a user-defined data type and behaves like built-in types of the programming language.

5. What is an encapsulation? (AUG'2013)*

Wrapping up of data and function within the structure is called as encapsulation. The insulation of data from direct access by the program is called as data hiding or information binding. The data is not accessible to the outside world and only those functions, which are wrapped in the class, can access it.

6. Write the process of programming in an object oriented language?

The processes of programming in an object-oriented language are,

- i. Create classes that define objects and their behavior.
- ii. Creating objects from class definition.
- iii. Establishing communication among objects.

7. What are Manipulators?

setw, endl are known as manipulators.

Manipulators are operators that are used to format the display. The endl manipulator when used in an output statement causes a linefeed to be inserted and its effect is similar to that of the newline character "\n".

Eg:

```
Cout<<setw(5)<<sum<<endl;
```

8. List any four advantages of OOPS

The advantages of OOPS are,

- i. The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.
- ii. It is possible to have multiple instances of an object to co-exist without any interference.
- iii. Object oriented programming can be easily upgraded from small to large systems.
- iv. Software complexity can be easily managed.

9. What are the features required for object based programming language?

The features required for object-based programming are,

- i. Data encapsulation.
- ii. Data hiding and access mechanisms.
- iii. Automatic initialization and clear up of objects.
- iv. Operator overloading.

10. Give any four applications of the OOPS.

The four applications of OOPS are,

- i. Real-time systems.
- ii. Simulation and modeling.
- iii. Object-oriented databases.
- iv. AI and expert systems.

11. How the class is specified?

Generally class specification has two parts. They are,

- i. Class declaration - It describes the type and scope of its member
- ii. Class function definition - It describes how the class functions are implemented

The general form is:

```
class class_name
{
private:
    variable declarations;
    function declaration;
public:
    variable declaration;
    function declaration;
};
```

12. How is the member functions defined?

Member functions can be defined in two ways. They are,

- i. **Outside the class definition** - Member function can be defined by using scope resolution operator (::).

General format is

return type class_ name:: function-name (argument declaration)

```
{  
.....//function body  
}
```

ii. **Inside the class definition** - This method of defining member function is to replace the function declaration by the actual function definition inside the class. It is treated as inline function

Example:

```
class item  
{  
int a, b;  
void getdata (int x, int y)  
{  
    a=x;  
    b=y;  
}};
```

13. What are the situations that inline functions may not work?

The situations that inline functions may not work is,

- i. For function returning values, if a loop, a switch, or a goto exists.
- ii. For function not returning values, if a return statement exists.
- iii. If function contains static variables.
- iv. If inline functions are recursive.

14. What are the features of static data member?(May 2007)

The features of static data member are,

- i. It is initialized to zero when the first object is created
- ii. Only one copy of that member is created for the entire class
- iii. It is only visible within the class
- iv. It is stored separately rather than objects

15. WHAT IS AN INLINE FUNCTION?

An inline function is a function that is expanded in line when it is invoked. Here, the compiler replaces the function call with the corresponding function code.

The inline function is defined as,

inline function-header

{

function body

}

16. What is data abstraction?

- The insulation of data from direct access by the program is called as data hiding or information binding.
- The data is not accessible to the outside world and only those functions, which are wrapped in the class, can access it.

17. What are data members and member functions?

Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight, and cost and uses functions to operate on these attributes. The attributes are sometimes called as data members because they hold information. The functions that operate on these data are called as methods or member functions.

Eg: `inta,b; // a,b are data members`

`Void getdata ( ) ; // member function`

18. What are Friend functions? Write the syntax

A function that has access to the private member of the class but is not itself a member of the class is called friend functions.

The general form is

`Friend data_typefunction_name( );`

Friend function is preceded by the keyword 'friend'.

19. Write some properties of friend functions.

- Friend function is not in the scope of the class to which it has been declared as friend. Hence it cannot be called using the object of that class.
- Usually it has object as arguments.
- It can be declared either in the public or private part of a class.
- It cannot access member names directly. It has to use an object name and dot membership operator with each member name. eg: (A . x)

20. What is the difference between structure and class?**Structures:-**

A hierarchical set of names that refer to an aggregate of data items may have different attributes.

Class:-

A group of objects that share common properties and relationships. A class is defined with a keyword class.

21. What are the two forms of if statement? Give the syntax also?

The two forms of if statement are

- i. Simple if statement
- ii. If – else statement

Syntax:-**Form 1:-**

If(expression is true)

```
{  
Action1;  
}  
Action2;
```

Form 2:-

If (expression is true)

```
{  
Action1;  
}  
Else  
{
```

```
Action2;  
}  
Action3;
```

22. What is a call by reference and call by value?

Call by reference: A function call mechanism that passes arguments to a function by passing the addresses of the arguments.

Call by value: A function call mechanism that passes arguments to a function by passing a copy of the value of the arguments.

23. Give the properties of the static member functions?

- i. A static function can have access to only other static members declared in the same class.
- ii. A static member function can be called using the class name as class name :: function name

24. What do you mean by enumerated datatype?

An enumerated datatype is another user-defined datatype, which provides a way for attaching names to numbers, thereby increasing comprehensibility of the code. The syntax of an enum statement is similar to that of the struct statement.

Eg:

```
enum shape{ circle, square, triangle}
```

```
enum color{ red, blue, green, yellow}
```

25. What is function overloading? Give an example

Function overloading means we can use the same function name to create functions that perform a variety of different tasks.

Eg: An overloaded add () function handles different data types as shown below.

// Declarations

i. int add(int a, int b); //add function with 2 arguments of same type

ii. int add(int a, int b, int c); //add function with 3 arguments of same type

iii. double add(int p, double q); //add function with 2 arguments of different type

//Function calls

add (3 , 4); //uses prototype (i.)

add (3, 4, 5); //uses prototype (ii.)

add (3 , 10.0); //uses prototype (iii.)

26. What is the return type of main ()?

- The return type of main () is integer i.e. main () returns an integer type value to the operating system.
- Therefore, every main () should end with a return (0) statement. It's general format is,

```
int main ()
```

```
{
```

```
.....
```

```
return 0;
```

```
}
```

27. State the use of void in C++.

The two normal uses of void are

- To specify the return type of the function when it is not returning a value
- To indicate an empty argument list to a function

Unit – I (11 marks)

1. Explain static member function with a program.

A member function that is declared static has the following properties:

- a) A static function can have access to only other static members (functions or variables) declared in the same class.
- b) A static member function can be called using the class name (instead of its objects) as follows:

Class-name :: function-name;

The program which is presented below illustrates the implementation of these characteristics. The static function showcount() displays the number of objects created till that moments. A count of number of objects creates is maintained by the static variable count.

The function showcode() displays the code number of each object.

```
#include <iostream>
using namespace std;
class test
{
    int code;
    static int count;           // static member variable
public:
    void setcode(void)
    {
        code = ++count;
    }
    void showcode(void)
    {
        cout << " object number: " << code << "\n";
    }
    static void showcount(void) // static member function
    {
        cout << "count:" << count << "\n";
    };
};
int test :: count;
int main()
{
    test t1,t2;
    t1.setcode();
    t2.setcode();
    test :: showcount();       // accessing static function
    test t3;
    t3.setcode();
    test :: showcount();
    t1.showcode();
    t2.showcode();
    t3.showcode();
    return 0;
}
```

Output of Program

```
count: 2
count: 3
object number: 1
object number: 2
object number: 3
```

2. Describe the basic concepts of structures with an example.

Structures are aggregate data types built using elements of other types.

```
Struct student
{
char name[20];
int roll number;
float marks;
}
```

The keyword struct introduces the structure definition. The identifier is the structure tag that names the structure definition and is used to declare variables of the structure type. The three fields of different data type are known as structure members or elements. The identifier students, which is referred to as structure name or structure tag, can be used to create variables of type student.

```
e.g
struct student A;                                //variable of new type student
```

```
e.g
#include<iostream.h>
```

```
struct time
{
int hour;
int minute;
int second;
};
```

```
void printmilitary (const time &);
void printstandard (const time &);
int main()
{
time t;
//set members to valid values
```

```

dinnertime.hour=18;
dinnertime.minute=30;
dinnertime.second=0;
cout<<"dinner will be held a";
printmilitary (dinnertime);
cout<<"military time, \n which is";
printstandard(dinnertime);
cout<<"standard time. \n";

        //set members to invalid value
dinnertime.hour=29;
dinnertime.minute=73;
cout<<"time with invalid values";
printmilitary(dinnertime);
cout<<endl;
return 0;
}

        //print the time in military format
voidprintmilitary (const time &t)
{
cout<< (t.hour<10 ? '0' : " ")<<t.hour<<":"<<(t.minute<10 ? '0' : " ")<<t.minute;

        //print the time in standard format
voidprintstandard (const time &t)
{
cout<<((t.hour ==0 ||t.hour ==12) ? 12 : t.hour % 12)<<":"<<(t.minute<10 ? "0" : " ")
<<t.minute<<":"<<(t.second<10 ? "0" : " ") <<t.second<<(t.hour<12 ? "AM":"PM");
}

```

O/P:

dinner will be held at 18.30 military time, which is 6:30:00 PM standard time

.

time with invalid values:29:73

3. Discuss about function in C++

Functions are the building blocks of C++ programs where all the program activity occurs. Function is a collection of declarations and statements.

Need for a Function

Monolithic program (a large single list of instructions) becomes difficult to understand. For this reason functions are used. A function has a clearly defined objective (purpose) and a clearly defined interface with other functions in the program. Reduction in program size is another reason for using functions. The functions code is stored in only one place in memory, even though it may be executed as many times as a user needs.

The following program illustrates the use of a function :

//to display general message using function

```
#include<iostream.h>
#include<conio.h>
void main()
{
    voiddisp();           //function prototype
    clrscr();             //clears the screen
    disp();               //function call
    getch();              //freeze the monitor
}

//function definition
voiddisp()
{
    cout<<"Welcome “ \n”";
    cout<<"Programming is nothing but logic implementation”;
}
```

Function Definition And Declaration

In C++, a function must be defined prior to its use in the program. The function definition contains the code for the function. The function definition for display_message () in program is given below the main () function. The general syntax of a function definition in C++ is shown below:

```
Type name_of_the_function (argument list)
{
    //body of the function
}
```

Here, the type specifies the type of the value to be returned by the function. It may be any valid C++ data type. When no type is given, then the compiler returns an integer value from the function.

Name_of_the_function is a valid C++ identifier (no reserved word allowed) defined by the user and it can be used by other functions for calling this function.

Argument list is a comma separated list of variables of a function through which the function may receive data or send data when called from other function. When no parameters, the argument list is empty as you have already seen in program 6.1. The following function illustrates the concept of function definition:

```
//function definition add ()
void add()
{
    int a,b,sum;
    cout<<"Enter two integers"<<endl;
    cin>>a>>b;
    sum=a+b;
    cout<<"\nThe sum of two numbers is "<<sum<<endl;
}
```

The above function add () can also be coded with the help of arguments of parameters as shown below:

```
//function definition add()
void add(int a, int b) //variable names are must in definition
{
    int sum;
    sum=a+b;
    cout<<"\nThe sum of two numbers is "<<sum<<endl;
}
```

Arguments to a Function

Arguments(s) of a function is (are) the data that the function receives when called/invoked from another function.

Passing Arguments to a Function

It is not always necessary for a function to have arguments or parameters. The functions add () and divide () in program 6.3 did not contain any arguments. The following example illustrates the concept of passing arguments to function SUMFUN ():

```
// demonstration of passing arguments to a function
#include<iostream.h>
void main ()
{
    float x,result; //local variables
    int N; //      formal parameters
    float SUMFUN(float x, int N); //function declaration Semicolon here
    result = SUMFUN(X,N); //function declaration

}
```

```
//function definition
SUMFUN( )
float SUMFUN(float x,int N) //function declaration
{ .....
.....
}
    No semicolon here
```

No semicolon here

Body of the function

Default Arguments

C++ allows a function to assign a parameter the default value in case no argument for that parameter is specified in the function call. For example.

```
// demonstrate default arguments function
#include<iostream.h>
int calc(int U)
{
    if (U % 2 == 0)
        return U+10;
    else
        return U+2
}
void pattern (char M, int B=2)
{
    for (int CNT=0;CNT<B; CNT++)
        cout<<calc(CNT) <<M;
        cout<<endl;
}
Void main ()
{
    Pattern('*');
    Pattern ('#',4)
    Pattern (;@,3);
}
```

Constant arguments

A C++ function may have constant arguments(s). These arguments(s) are/are treated as constant(s). These values cannot be modified by the function. For making the arguments(s) constant to a function, we should use the keyword const as given below in the function prototype :

```
void max(const float x, const float y, const float z);
```

Here, the qualifier const informs the compiler that the arguments(s) having const should not be modified by the function max (). These are quite useful when call by reference method is used for passing arguments.

Calling Functions

In C++ programs, functions with arguments can be invoked by :

(a) Value (b) Reference

Call by Value: - In this method the values of the actual parameters (appearing in the function call) are copied into the formal parameters (appearing in the function definition), i.e., the function creates its own copy of argument values and operates on them. The following program illustrates this concept :

```
//calculation of compound interest using a function
#include<iostream.h>
#include<conio.h>
#include<math.h> //for pow()function
void main()
{
    float principal, rate, time; //local variables
    void calculate (float, float, float); //function prototype clrscr();
    cout<<"\nEnter the following values:\n";
    cout<<"\nPrincipal:";
    cin>>principal;
    cout<<"\nrate of interest:";
    cin>>rate;
    cout<<"\ntime period (in yeasers) :";
    cin>>time;
    calculate (principal, rate, time); //function call
    getch ();
}
//function definition calculate()
void calculate (float p, float r, float t)
{
    float interest; //local variable
    interest = p* (pow((1+r/100.0),t))-p;
    cout<<"\ncompound interest is : "<<interest;
}
```

Call by Reference: -

A reference provides an alias – an alternate name – for the variable, i.e., the same variable's value can be used by two different names : the original name and the alias name. In call by reference method, a reference to the actual arguments(s) in the calling program is passed (only variables). So the called function does not create its own copy of original value(s) but works with the original value(s) with different name. Any change in the original data in the called function gets reflected back to the calling function.

It is useful when you want to change the original variables in the calling function by the called function.


```
//Swapping of two numbers using function call by reference
#include<iostream.h>
#include<conio.h>
void main()
{
    clrscr();
    int num1,num2;
    void swap (int&, int&); //function prototype
    cin>>num1>>num2;
    cout<<"\nBefore swapping:\nNum1: "<<num1;
    cout<<endl<<"num2: "<<num2;
    swap(num1,num2); //function call
    cout<<"\n\nAfter swapping : \Num1: "<<num1;
    cout<<endl<<"num2: "<<num2;
    getch();
}
//function definition
void swap (int& a, int& b)
{
    int temp=a;
    a=b;
    b=temp;
}
```

4. Give detailed explanation for manipulators.

The output formats can be controlled using manipulators. The header file `iomanip.h` has a set of functions. Effect of these manipulators is the same as `ios` class member functions. Every `ios` member function has two formats. One is used for setting and second format is used to know the previous setting. But the manipulator does not return to the previous setting. The manipulator can be used with `cout()` statement as given below.

```
Cout<<m1<<m2<<v1;
```

Here `m1` and `m2` are two manipulators and `v1` is any valid C++ variable.

Table describes the most useful manipulators. The manipulator `hex`, `dec`, `oct`, `ws`, `endl` and `flush` are defined in `iostream.h`. The manipulator `setbase`, `width()`, `fill()` etc. that requires an argument are defined in `iomanip.h`.

Table : Pre-defined manipulators

Manipulator	Function
-------------	----------

setw(int n)	The field width is fixed to n.
setbase	Sets the base of the number system.
setprecision(int p)	The precision point is fixed to p.
setfill(char f)	The fill character is set to character stored in variable f.
setiosflags(long 1)	Format flag is set to 1
resetiosflags(long 1)	Removes the flags indicated by 1.
endl	Splits a new line.
skipws	Omits white space on input.
noskipws	Does not omit white space on input.
ends	Adds null character to close an output string.
flush	Flushes the buffer stream.
lock	Locks the file associated with the file handle.
ws	Used to omit the leading white spaces present before the first
hex, oct, dec	Displays the number in hexadecimal, octal and decimal format.

program to display formatted output using manipulator.

```
#include <iostream.h>
#include<iomanip.h>
#include<conio.h>
{
    clrscr();
    cout<<setw(10) <<"hello"<<endl;
    cout<<setw(15) <<setprecision(2) << 2.5555;
    cout<<setiosflags(ios::hex);
    cout<<endl<< " hexadecimal of 84 is : "<<84;
    return 0;
}
```

OUTPUT

Hello

2.56

Hexadecimal of 84 is: 54

Explanation: In the above program, the manipulator setw(10) sets the field width to 10. The message “hello” is displayed to column 10. The endl inserts a new line. In the second cout() statement, the setprecision(2) manipulator sets the decimal point to 2. The number 2.5555 will be displayed as 2.56 at column 15. The manipulator setiosflag sets the hexadecimal setting for the display of number. The last cout() statement displays the equivalent hexadecimal of 84 i.e. 54.

Program to display the given decimal number in hexadecimal and octal format.

```
#include <iostream.h>
#include <conio.h>
#include <iomanip.h>
void main()
{
    clrscr();
    int x = 84;
    cout<< “\n Hexa- decimal number : “<<hex<<x;
    cout<< “\n Octal number : “<<oct<< x;
}
```

OUTPUT

Hexadecimal number: 54

Octal Number: 124

Explanation: In the above program, the integer variable x is declared and initialized with 84. The first cout statement displays the decimal number to its equivalent hexadecimal number. The manipulator hex converts decimal number to its hexadecimal equivalent. The second cout statement converts decimal number to its equivalent octal number.

2.43 Write a program to read number in hexadecimal format using cin statement. Display the number in decimal format.

```
# include <iostream.h>
# include <conio.h>

void main()
{
    clrscr();
    int x;
    cout <<"\n Enter Hexa-decimal Number : ";
    cin >>hex>>x;
    cout <<"\n Decimal Number      : "<<dec<<x;
}
```

OUTPUT

Enter Hexadecimal Number : 31

Decimal Number : 25

Explanation: In the above program, the cin statement reads a number in hex format. The cout statement displays its equivalent decimal number with the use of dec manipulator.

2.44 Write a program to demonstrate the use of endl manipulator.

```
# include <iostream.h>
# include <conio.h>

void main()
{
    clrscr();
    cout <<" Demo of endl ";
    endl(cout);
    cout <<" It splits a line" ;
}
```

OUTPUT

Demo of endl

It splits a line

Explanation: In the above program, endl manipulator is used to split the line. The two strings are displayed on two separate lines.

2.45 Write a program to demonstrate the use of flush() statement.

```
# include <iostream.h>
# include <conio.h>

void main()
{
    char text[20];
    clrscr();

    cout <<"Enter text  : ";
    cin.getline(text,20);
    cout<<"Text Entered : "<<text;
    flush(cout);
}
```

OUTPUT**Enter text : Buffer****Text Entered : Buffer**

Explanation: In the above program, the statement flush(cout) flushes the buffer. This statement can be used as cout<<flush.

5. Explain object oriented programming concepts.

There are several fundament concepts in object-oriented programming. Some of these are shown in fig.1.9 and are discussed below:

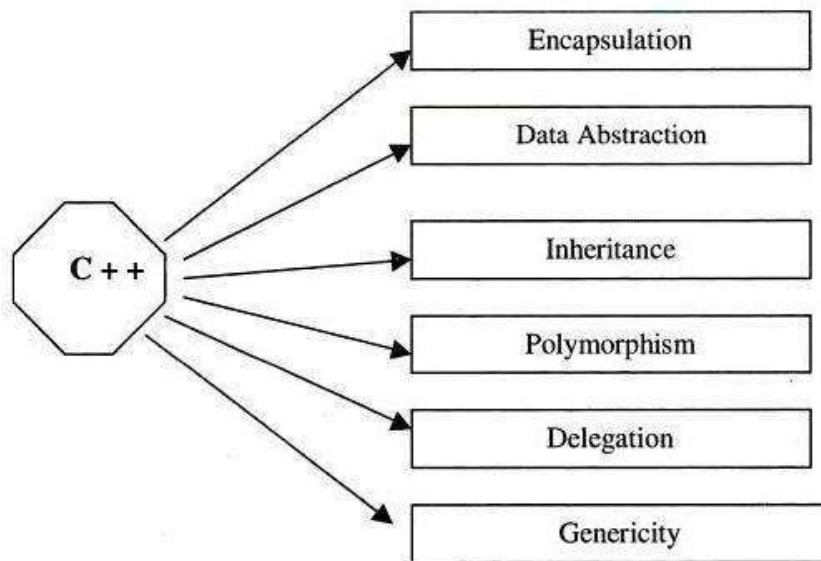


Fig. 1.9 Features of object-oriented programming

Objects

Objects are primary run-time entities in an object oriented programming. They may stand for a thing that has specific application for example, a spot, a person, any data item related to program, including user-defined data types. Programming issues are analyzed in terms of object and the type of transmission between them. Figure 1.10 describes various objects. The selected program objects must be similar to actual world objects. Objects occupy space in memory. Every object has its own properties or features. An object is a specimen of a class. It can be singly recognized by its name. it declares the state shown by the data values of its characteristic at a specific time. The state of the object varies according to procedure used. It is known as the action of the object. The action of the object depends upon the member function defined within it class.

Given below are some objects that we use in our daily life.



Fig. 1.10 Commonly available objects

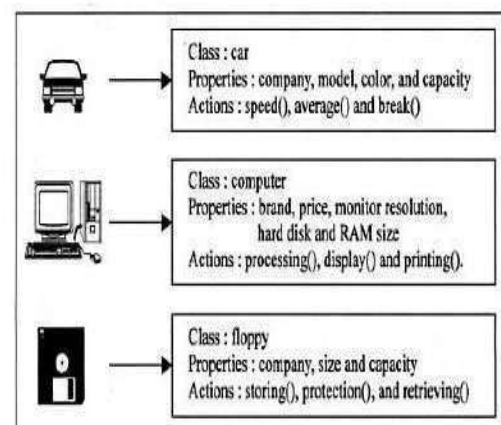


Fig. 1.11 Objects and their properties

Classes

A class is the accomplishment of abstract data type. It defines the nature and methods that act on the data structure and abstract data type respectively. Specimens are also known as objects. In other words, a class is a grouping of objects having identical properties, common behavior and shared relationship. The entire group of data and code of an object can be built as a user defined data type using class. Fig.1 .12 describes different classes and related data that come under them.

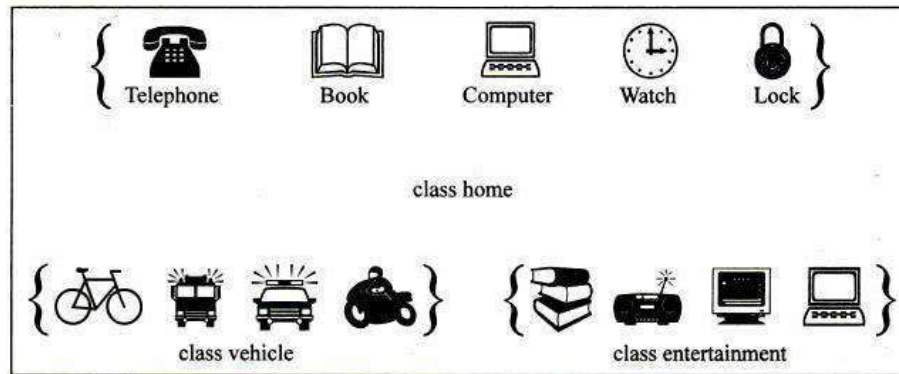


Fig. 1.12 Classes and their members

Objects are nothing but variables of type class. Once a class has been declared, the programmer can create a number of objects associated with that class. The syntax used to create an object is similar to the syntax used to create an integer variable in C. a class is a model of the object. Every object has its own value for each of its member variables. However, it shares the property names of operations with other instances of the class. Thus, classes define the characteristics and actions of different objects.

In fig. 1.11 some common objects, their properties, and the actions they perform have been described. A car can be distinguished from another car on the basis of its properties, for example, company, model, color, capacity etc. and also on the basis of its actions for example, speed, average, break etc.

Method

An operation required for an object or entity when coded in a class is called a method. The operations that are required for an object are to be defined in a class. All objects in a class perform certain common actions or operations. Each action needs an object that becomes a function of the class that defines it and is referred to as a method.

In fig.1.13, the class, its associated data members, and functions are shown in different styles. This style is frequently used while writing a class in the program.

The class A contains private data members and public methods or member functions. Generally, the data members are declared private, and member functions are declared public. We can also define public data members and private member functions. The data member of any class uses its member functions or methods to perform operations.

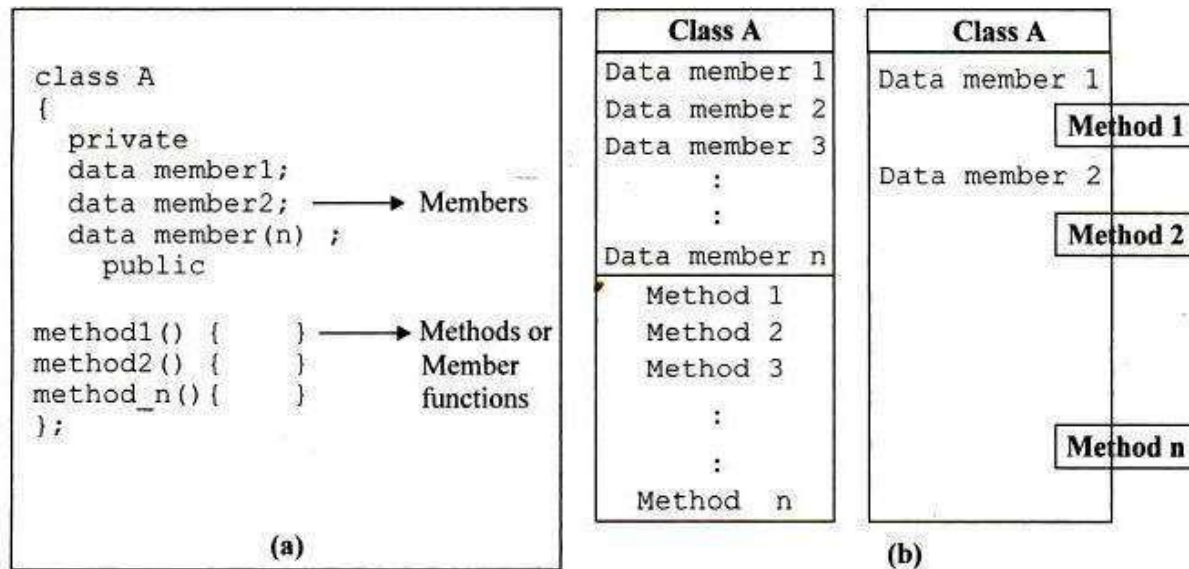


Fig. 1.13 Representation of methods in different manners

Data Abstraction

Abstraction directs the procedure of representing essential features without including the background details. Classes use the theory of abstraction and are defined as a list of abstract properties such as size, cost, height, and few functions to operate on these properties. Data abstraction is the procedure of identifying properties and methods related to a specific entity as applicable to the application.

A powerful method to achieve abstraction is by the manipulation of hierarchical classifications. It permits us to break the semantics of multiple systems into layer, by separating them into multiple controllable parts. For example, a computer as shown in fig 1.14 is made of various parts such as cpu, keyboard and so on. We think it as a single unit, but it has several sub units. These units when combined together perform the required task. By assembling subparts we can make a single system.

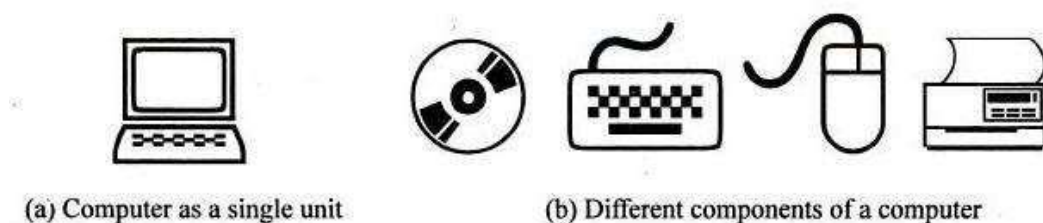


Fig. 1.14 Computer and its parts

Hierarchical abstraction of complicated systems can also be used in computer software. The data from conventional procedure oriented programs can be converted by abstraction mechanism into

its partial objects. A series of operation steps may develop a set of messages between these objects. Each object shows its own attributes.

Encapsulation

C++ supports the features of encapsulation using classes. The packing of data and functions into a single component is known as encapsulation. The data is not accessible by outside functions. Only those functions that are able to access the data are defined within the class. These functions prepare the interface between the object's data and the program. With encapsulation data hiding can be accomplished. Data hiding is an important feature. By data hiding an object can be used by the user without knowing how it works internally.

In C++ the fundamental of encapsulation is class. A class defines the structure of data and member functions. It is common to all its objects. Class is a logical structure whereas an object is a physical quantity. The goal of the class is to encapsulate complication. The class also has mechanism for hiding the data. Each member in the class may be private or public. Any non-member function cannot access the data of the class. The public section of the class must be carefully coded not to expose the inner details of the class. Fig. 1.15 explains different sections of encapsulation.

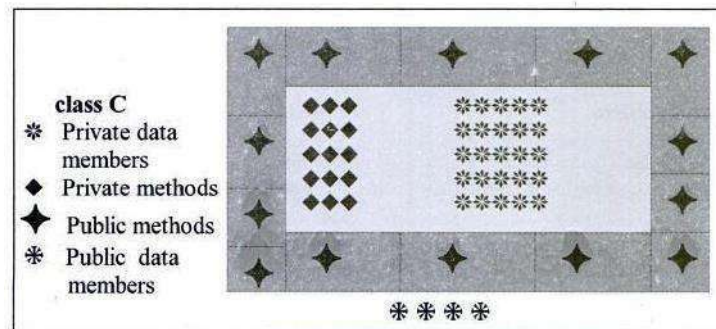
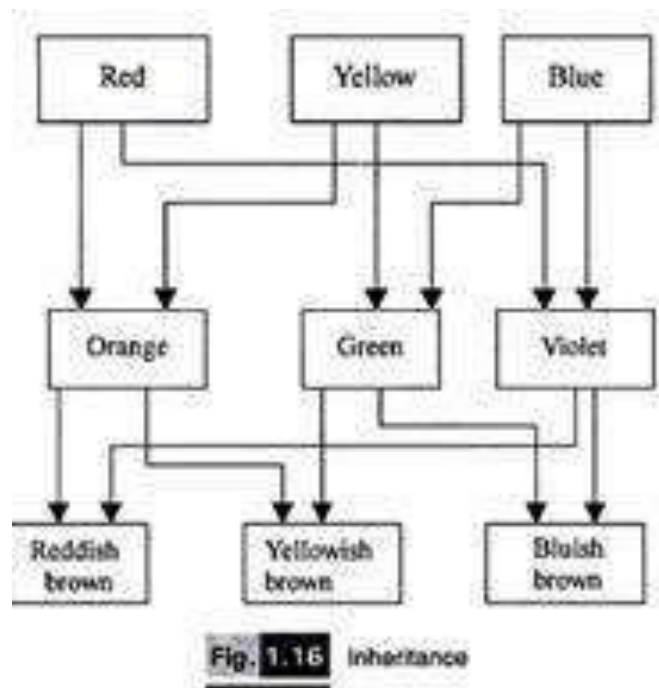


Fig. 1.15 Encapsulation: Private and Public sections

Inheritance

Inheritance is the method by which objects of one class get the properties of objects of another class. In object oriented programming, inheritance provides the thought of reusability. The programmer can add new properties to the existing class without changing it. This can be achieved by deriving a new class from the existing one. The new class will possess features of both the classes. The actual power of inheritance is that it permits the programmer to reuse a class that is close to what he wants and to tailor the class in such a manner that it does not bring any unwanted incidental result into the rest of the class. Thus, inheritance is the feature that permits the reuse of an existing class to make a new class. The fig. 1.16 shows an example of inheritance.

In fig 1.16 red, yellow and blue are the main colours. Orange colour is created from the combination of red and yellow, green is created from yellow and blue, and violet is created from red and blue. Orange colour has attributes of both red and yellow, which produces a new effect thus, many combinations are possible.



Polymorphism

Polymorphism is another important OOP concept. Polymorphism, a Greek term, means the ability to take more than one form. An operation may exhibit different behaviours in different instances. The behavior depends upon the types of data used in the operation. For example, consider the operation of addition. For two numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation. The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.

Fig 1.9 illustrates that a single function name can be used to handle different number and different types of arguments. This is something similar to a particular word having several different meanings depending on the context. Using a single function name to perform different types of tasks is known as function overloading.

Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface. This means that a general class of operations may be accessed in the same manner even though specific actions associated with each operation may differ. Polymorphism is extensively used in implementing inheritance.

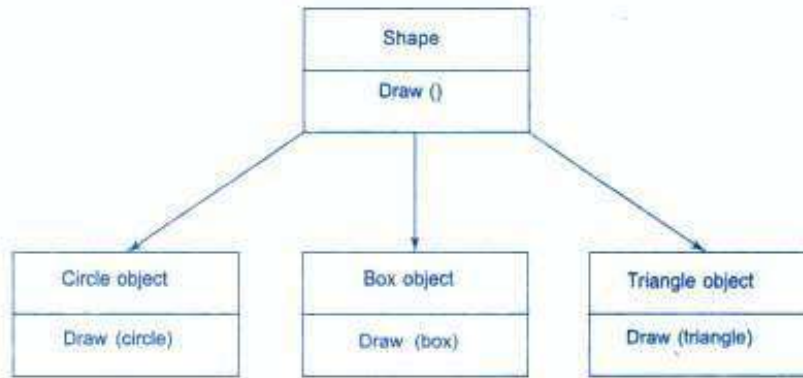


Fig.19 ⇔ Polymorphism

Dynamic Binding

Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding also known as late binding means that the code associated with a given procedure call is not known until the time of the call at run-time. It is associated with polymorphism and inheritance. A function call associated with a polymorphic reference depends on the dynamic type of that reference.

Message passing

An object oriented program consists of a set of objects that communicate with each other. The process of programming in an object oriented language, therefore, involves the following basic steps:

1. Creating classes that define objects and their behaviours,
2. Creating objects from class definitions, and
3. Establishing communication among objects.

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another. The concept of message passing makes it easier to talk about building systems that directly model or simulate their real-world counterparts. A message for an object is a request for execution of a procedure, and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function(message) and the information to be sent. Example



Objects have a life cycle. They can be created and destroyed. Communication with an object is feasible as long as it is alive.

Genericity

The software components of a program have more than one version depending on the data types of arguments. This feature allows declaration of variable without specifying exact data type. The compiler identifies the data type at run-time. The programmer can create a function that can be used for any type of data. The template feature in C++ allows generic programming.

6. Discuss about array of objects in C++

We know that an array can be of any data type including struct. Similarly, we can also have arrays of variables that are of the type class. Such variables are called array of objects. Consider the following class definition:

```
class employee
{
    char name[30];
    float age;
public:
    void getdata(void);
    void putdata(void);
};
```

The identifier employee is a user-defined data type and can be used to create objects that relate to different categories of the employees. Example:

```
employee manager[3]; // array of manager
employee foreman[15]; // array of foreman
employee worker[75]; // array of worker
```

The array manager contains three objects (managers), namely, manager [0], manager [1] and manager [2], of type employee class. Similarly, the foreman array contains 15 objects and the worker array contains 75 objects.

Since an array of objects behaves like any other array, we can use the usual array accessing methods to access individual elements, and then the dot member operator to access the member functions. For example, the statement

```
manager[i].putdata();
```

Will display the data of the ith element of the array manager. That is, this statement requests the object manager[i] to invoke the member function putdata().

An array of objects is stored inside the memory in the same way as a multi-dimensional array. The array manager is represented in fig. note that only the space for data items of the objects is created. Member functions are stored separately and will be used by all the objects.



Fig. 5.5 ⇔ Storage of data items of an object array

Program 5.6 illustrates the use of object arrays.

ARRAYS OF OBJECTS

```
#include <iostream>
using namespace std;
class employee
```

```
{
    char name[30];    // string as class member
    float age;
public:
    void getdata(void);
    void putdata(void);
};
void employee :: getdata(void)
{
    cout << "Enter name: ";
    cin >> name;
    cout << "Enter age: ";
    cin >> age;
}
void employee :: putdata(void)
{
    cout << "Name: " << name << "\n";
    cout << "Age: " << age << "\n";
}
const int size=3;
int main()
{
    employee manager[size];
    for(int i=0; i<size; i++)
    {
        cout << "\nDetails of manager" << i+1 << "\n";
        manager[i].getdata();
    }
    cout << "\n";
    for(i=0; i<size; i++)
    {
        cout << "\nManager" << i+1 << "\n";
        manager[i].putdata();
    }
    return 0;
}
```

PROGRAM 5.6

This being an interactive program, the input data and the program output are shown below:

Interactive input

```
Details of manager1
Enter name: xxx
Enter age: 45
```

```
Details of manager2  
Enter name: yyy  
Enter age: 37
```

```
Details of manager3  
Enter name: zzz  
Enter age: 50
```

Program output

```
Manager1  
Name: xxx  
Age: 45
```

```
Manager2  
Name: yyy  
Age: 37
```

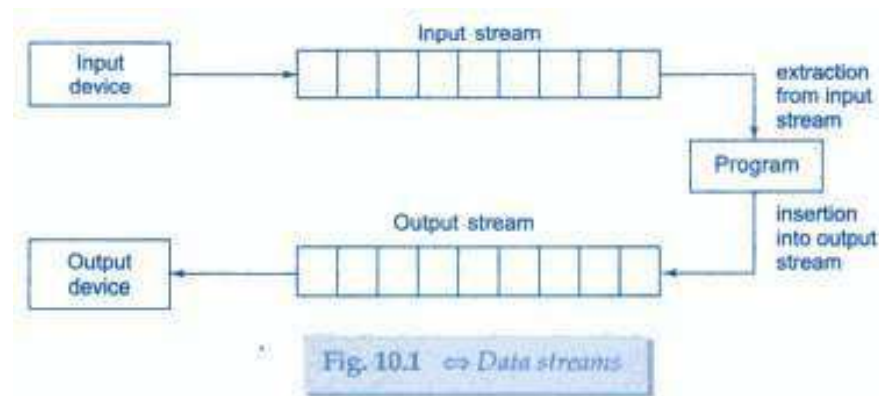
```
Manager3  
Name: zzz  
Age: 50
```


7. Explain the different types of streams and various formatted I/O in C++.

C++ Streams

The I/O system in C++ is designed to work with a wide variety of devices including terminals, disks and tape drives. Although each device is very different, the I/O system supplies an interface to the programmer that is independent of the actual device being accessed. This interface is known as stream.

A stream is a sequence of bytes. It acts either as a source from which the input data can be obtained or as a destination to which output data can be sent. The source stream that provides data to the program is called the input stream and the destination stream that receives output from the program is called the output stream. In other words, a program extracts the bytes from an input stream and inserts bytes into an output stream as illustrated in figure.



The data in input stream can come from the keyboard or any other storage device. Similarly, the data in the output stream can go to the screen or any other storage device. C++ contains several predefined streams that are automatically opened when a program begins its execution. These include `cin` and `cout`. `cin` represents the input stream connected to the standard input device and `cout` represents the output stream connected to the standard output device.

C++ stream classes

The C++ I/O system contains a hierarchy of classes that are used to define various streams to deal with both the console and disk files. These classes are called stream classes. Fig shows the hierarchy of the stream classes used for input and output operations with the console unit. These classes are declared in the header file `iostream`. This file should be included in all the programs that communicate with the console unit.

As seen in fig `ios` is the base class for `istream`(input stream) and `ostream`(output stream) which are, in turn base classes for `iostream`(input/output stream). The class `ios` is declared as the virtual base class so that only one copy of its members are inherited by the `iostream`.



Fig. 10.2 ↔ *Stream classes for console I/O operations*

The class `ios` provides the basic support for formatted and unformatted I/O operations. The class `istream` provides the facilities for formatted and unformatted input while the class `ostream` provides the facilities for formatted output. The class `iostream` provides the facilities for handling both input and output streams. Three classes namely, `istream_withassign`, `ostream_withassign` and `iostream_withassign` add assignment operators to these classes.

Table 10.1 Stream classes for console operations

Class name	Contents
ios (General input/output stream class)	▪ Contains basic facilities that are used by all other input and output classes ▪ Also contains a pointer to a buffer object (streambuf object) ▪ Declares constants and functions that are necessary for handling formatted input and output operations
istream (input stream)	▪ Inherits the properties of ios ▪ Declares input functions such as get(), getline() and read() ▪ Contains overloaded extraction operator >>
ostream (output stream)	▪ Inherits the properties of ios ▪ Declares output functions put() and write() ▪ Contains overloaded insertion operator <<
iostream (input/output stream)	▪ Inherits the properties of ios, istream and ostream through multiple inheritance and thus contains all the input and output functions
streambuf	▪ Provides an interface to physical devices through buffers ▪ Acts as a base for filebuf class used ios files

Formatted console I/O operations

C++ supports a number of features that could be used for formatting the output. These features include:

- ios class functions and flags
- Manipulators
- User-defined output functions

The ios class contains a large number of member functions that would help us to format the output in a number of ways. The most important ones among them are listed in table.

Table 10.2 ios format functions

Function	Task
Width ()	To specify the required field size for displaying an output value
precision ()	To specify the number of digits to be displayed after the decimal point of a float value
fill()	To specify a character that is used to fill the unused portion of a field
setf()	To specify format flags that can control the form of output display (such as left-justification and right-justification)
unsetf()	To clear the flags specified

Manipulators are special functions that can be included in the I/O statements to alter the format parameters of a stream. Table shows some important manipulator functions that are frequently used. To access these manipulators, the file iomanip should be included in the program.

Table 10.3 Manipulators

Manipulators	Equivalent ios function
setw()	width()
setprecision()	precision()
setfill()	fill()
setiosflags()	setf()
resetiosflags()	unsetf()

Defining Field width: width()

We can use the width() function to define the width of a field necessary for the output of an item. Since it is a member function, we have to use an object to invoke it, as shown below:

```
cout.width(w);
```

where w is the field width. The output will be printed in a field of w characters wide at the right end of the field. The width() function can specify the field width for only one item.

After printing one item it will revert back to the default. For example, the statements

```
cout.width(5);
```

```
cout<< 543 << 12 <<"\n";
```

will produce the following output:

		5	4	3	1	2
--	--	---	---	---	---	---

The value 543 is printed right-justified in the first five columns. The specification width(5) does not retain the setting for printing the number 12. This can be improved as follows:

```
cout.width(5);  
cout<< 543;  
cout.width(5);  
cout<< 12 << "\n";
```

This produces the following output:

		5	4	3				1	2
--	--	---	---	---	--	--	--	---	---

SPECIFYING FIELD SIZE WITH width()

```

#include <iostream>
using namespace std;

int main()
{
    int items[4] = {10,8,12,15};
    int cost[4] = {75,100,60,99};

    cout.width(5);
    cout << "ITEMS";
    cout.width(8);
    cout << "COST";

    cout.width(15);
    cout << "TOTAL VALUE" << "\n";

    int sum = 0;

    for(int i=0; i<4; i++)
    {
        cout.width(5);
        cout << items[i];

        cout.width(8);
        cout << cost[i];

        int value = items[i] * cost[i];
        cout.width(15);
        cout << value << "\n";
        sum = sum + value;
    }
    cout << "\n Grand Total = ";

    cout.width(2);
    cout << sum << "\n";

    return 0;
}

```

PROGRAM 10.4

The output of Program 10.4 would be:

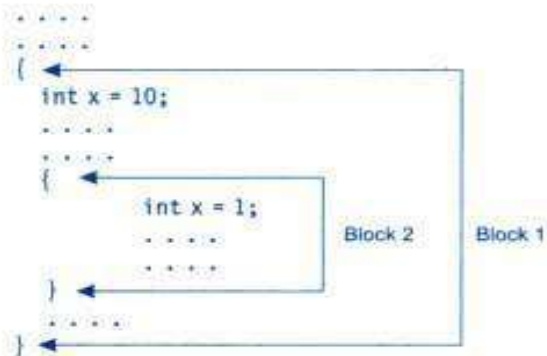
ITEMS	COST	TOTAL VALUE
10	75	750
8	100	800
12	60	720
15	99	1485
Grand Total =		3755

8. Explain Scope resolution operator with an example

Like C, C++ is also a block-structured language. Blocks and scopes can be used in constructing programs. The same variable name can be used to have different meanings in different blocks. The scope of the variable extends from the point of its declaration till the end of the block containing the declaration. A variable declared inside a block is said to be local to that block. Consider the following segment of a program:

```
.....  
.....  
{  
    int x = 10;  
    .....  
    .....  
}  
.....  
.....  
{  
    int x = 1;  
    .....  
    .....  
}
```

The two declarations of `x` refer to two different memory locations containing different values. Statements in the second block cannot refer to the variable `x` declared in the first block, and vice versa. Blocks in C++ are often nested. For example, the following style is common:



Block2 is contained in block1. Note that a declaration in an inner block *hides* a declaration of the same variable in an outer block and, therefore, each declaration of `x` causes it to refer to

a different data object. Within the inner block, the variable **x** will refer to the data object declared therein.

In C, the global version of a variable cannot be accessed from within the inner block. C++ resolves this problem by introducing a new operator **::** called the *scope resolution operator*. This can be used to uncover a hidden variable. It takes the following form:

```
:: variable-name
```

This operator allows access to the global version of a variable. For example, **::count** means the global version of the variable **count** (and not the local variable **count** declared in that block). Program 3.1 illustrates this feature.

SCOPE RESOLUTION OPERATOR

```
#include <iostream>

using namespace std;
int m = 10;           // global m

int main()
{
    int m = 20;       // m redeclared, local to main
    {
        int k = m;
        int m = 30;   // m declared again
                     // local to inner block

        cout << "we are in inner block \n";
        cout << "k = " << k << "\n";
        cout << "m = " << m << "\n";
        cout << "::m = " << ::m << "\n";
    }

    cout << "\nWe are in outer block \n";
    cout << "m = " << m << "\n";
    cout << "::m = " << ::m << "\n";

    return 0;
}
```

PROGRAM 3.1

The output of Program 3.1 would be:

```
We are in inner block
k = 20
```

```
m = 30
::m = 10

We are in outer block
m = 20
::m = 10
```

In the above program, the variable **m** is declared at three places, namely, outside the **main()** function, inside the **main()**, and inside the inner block.

note

It is to be noted **::m** will always refer to the global **m**. In the inner block, **::m** refers to the value 10 and not 20.

A major application of the scope resolution operator is in the classes to identify the class to which a member function belongs. This will be dealt in detail later when the classes are introduced.

9. With an example discuss about friend function

C++ Friend Functions

Need for Friend Function As discussed in the earlier sections on access specifiers, when a data is declared as private inside a class, then it is not accessible from outside the class. A function that is not a member or an external class will not be able to access the private data. A programmer may have a situation where he or she would need to access private data from non-member functions and external classes. For handling such cases, the concept of Friend functions is a useful tool.

What is a Friend Function? A friend function is used for accessing the non-public members of a class. A class can allow non-member functions and other classes to access its own private data, by making them friends. Thus, a friend function is an ordinary function or a member of another class.

How to define and use Friend Function in C++?

The friend function is written as any other normal function, except the function declaration of these functions is preceded with the keyword **friend**. The friend function must have the class to which it is declared as friend passed to it in argument.

Some important points to note while using friend functions in C++:

- The keyword **friend** is placed only in the function declaration of the friend function and not in the function definition. It is possible to declare a function as friend in any number of classes.
- When a class is declared as a friend, the friend class has access to the private data of the class that made this a friend.

- A friend function, even though it is not a member function, would have the rights to access the private members of the class. It is possible to declare the friend function as either private or public.
- The function can be invoked without the use of an object. The friend function has its argument as objects, seen in example below. Example to understand the friend function:

```
#include<iostream.h>
using namespace std;
class sample
{
int a;
int b;
public:
void setvalue() {a=25; b=40; }
friend float mean(sample s);
};
float mean(sample s)
{
return float(s.a+s.b)/2.0;
}
int main()
{
sample x;
x.setvalue();
cout<<"mean value="<<mean(x)<<"\n";
return 0;
}
o/p:mean value: 32.5
```

The friend function accesses the class variables a & b by using the dot operator and the object passed to it. The function call mean(x) passes the object x by value to the friend function.

Member function of one class can be friend functions of another class. In such cases they are defined using the scope resolution operator as shown below:

```
class x
{
...
.....
int func1();
...
};
class y
{
```

```

.....
friendint x :: int func1();          //func1() of x is friend of y
...
};

```

We can also declare all the member functions of one class as the friend functions of another class. In such cases, the class is called a friend class.

```

Class z
{
...
friend class x;
};

```

FUNCTION FRIENDLY TO TWO CLASS

```

#include<iostream.h>
using namespace std;
class Abc;
class Xyz
{
int x;
public:
void setvalue(int i) {x=i;}
friend void max(Xyz, Abc);
};
class Abc
{
int a;
public:
void setvalue(int i) {a=i;}
friend void max(Xyz, Abc);
};
void max(Xyz m, Abc n)
{
if(m.x>=n.a)
cout<<m.x;
else
cout<<n.a;
}
int main()
{
Abc abc;
abc.setvalue(10);
Xyz xyz;
xyz.setvalue(20);
max(xyz, abc);
return 0;
}

```

o/p:
20

The function max() has arguments from both Xyz and Abc. When the function max() is declared as a friend in Xyz for the first time, the compiler will not acknowledge the presence of Abc unless its name is declared in the beginning as

```
classabc;
```

This is known as forward declaration.

10. Discuss in detail about inline function.

These are the functions designed to speed up program execution. An inline function is expanded (i.e. the function code is replaced when a call to the inline function is made) in the line where it is invoked. You are familiar with the fact that in case of normal functions, the compiler have to jump to another location for the execution of the function and then the control is returned back to the instruction immediately after the function call statement. So execution time taken is more in case of normal functions. There is a memory penalty in the case of an inline function.

The system of inline function is as follows :

```
Inlinefunction_header
```

```
{
    body of the function
}
```

For example,

```
//function definition min()
```

```
inline void min (int x, int y)
```

```
{
    cout<< (x < Y? x : y);
}
```

```
void main()
```

```
{
    int num1, num2;
    cout<<"\nEnter the two intergers\n";
    cin>>num1>>num2;
    min (num1,num2); //function code inserted here
    -----
    -----
}
```

An inline function definition must be defined before being invoked as shown in the above example. Here min () being inline will not be called during execution, but its code would be inserted into main () as shown and then it would be compiled.

If the size of the inline function is large then heavy memory penalty makes it not so useful and in that case normal function use is more useful.

The inline does not work for the following situations:

1. For functions returning values and having a loop or a switch or a goto statement.
2. For functions that do not return value and having a return statement.
3. For functions having static variable(s).
4. If the inline functions are recursive (i.e. a function defined in terms of itself).

The benefits of inline functions are as follows:

1. Better than a macro.
2. Function call overheads are eliminated.
3. Program becomes more readable.
4. Program executes more efficiently.

INLINE FUNCTIONS

```
#include <iostream>

using namespace std;

inline float mul(float x, float y)
{
    return(x*y);
}

inline double div(double p, double q)
{
    return(p/q);
}

int main()
{
    float a = 12.345;
    float b = 9.82;

    cout << mul(a,b) << "\n";
    cout << div(a,b) << "\n";

    return 0;
}
```

PROGRAM 4.1

The output of program 4.1 would be

```
121.228
1.25713
```